EXPERIMENT 1: STUDENT–COLLEGE DATABASE (BASIC SETUP)

MySQL 8.0+ • Beginner-friendly example • Run in MySQL Workbench

```
-- Experiment 1: Create and select the Student–College database
CREATE DATABASE college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL UNIQUE
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

INSERT INTO department VALUES
(1, 'Computer'),
(2, 'Information Technology');

INSERT INTO student VALUES
(101, 'Amit', 20, 1),
(102, 'Neha', 21, 2),
(103, 'Rahul', 19, 1);

SELECT * FROM department;
SELECT * FROM student;
```

EXPERIMENT 2: ER/EER TO RELATIONAL SCHEMA

MySQL 8.0+ • Beginner-friendly example • Run in MySQL Workbench

```
-- Experiment 2: Relational schema for Student–College case study
CREATE DATABASE college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL UNIQUE
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    age INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

CREATE TABLE course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL,
    credits INT
);

CREATE TABLE enrollment (
    roll_no INT,
    course_id INT,
    marks INT,
    PRIMARY KEY (roll_no, course_id),
    FOREIGN KEY (roll_no) REFERENCES student(roll_no),
    FOREIGN KEY (course_id) REFERENCES course(course_id)
);

-- Relational schema:
-- DEPARTMENT(dept_id PK, dept_name)
-- STUDENT(roll_no PK, student_name, age, dept_id FK)
-- COURSE(course_id PK, course_name, credits)
-- ENROLLMENT(roll_no PK/FK, course_id PK/FK, marks)
```

EXPERIMENT 3: DDL AND INTEGRITY CONSTRAINTS

MySQL 8.0+ • Beginner-friendly example • Run in MySQL Workbench

```
-- Experiment 3: DDL and integrity constraints
CREATE DATABASE college;
USE college;

CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL UNIQUE
);

CREATE TABLE student (
    roll_no INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    email VARCHAR(60) UNIQUE,
    age INT CHECK (age >= 16),
    dept_id INT,
    status VARCHAR(15) DEFAULT 'Active',
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

ALTER TABLE student ADD phone VARCHAR(15);
ALTER TABLE student MODIFY student_name VARCHAR(60);
ALTER TABLE student DROP COLUMN phone;

CREATE TABLE student_test (
    id INT PRIMARY KEY,
    name VARCHAR(30) NOT NULL
);

RENAME TABLE student_test TO learner_test;
TRUNCATE TABLE learner_test;
DROP TABLE learner_test;

SHOW TABLES;
```

EXPERIMENT 4: DML (INSERT, UPDATE, DELETE)

MySQL 8.0+ • Beginner-friendly example • Run in MySQL Workbench

```
-- Experiment 4: DML commands
USE college;

INSERT INTO department (dept_id, dept_name)
VALUES (1, 'Computer'), (2, 'IT');

INSERT INTO student (roll_no, student_name, email, age, dept_id)
VALUES
(101, 'Amit', 'amit@example.com', 20, 1),
(102, 'Neha', 'neha@example.com', 21, 2),
(103, 'Rahul', 'rahul@example.com', 19, 1);

SELECT * FROM student;

UPDATE student
SET age = 22
WHERE roll_no = 102;

SELECT * FROM student;

DELETE FROM student
WHERE roll_no = 103;

SELECT * FROM student;
```

EXPERIMENT 5: DCL (GRANT AND REVOKE)

MySQL 8.0+ • Beginner-friendly example • Run in MySQL Workbench

```
-- Experiment 5: DCL commands
CREATE USER 'college_user'@'localhost'
IDENTIFIED BY 'College123';

GRANT SELECT, INSERT, UPDATE
ON college.student
TO 'college_user'@'localhost';

SHOW GRANTS FOR 'college_user'@'localhost';

REVOKE INSERT, UPDATE
ON college.student
FROM 'college_user'@'localhost';

SHOW GRANTS FOR 'college_user'@'localhost';

-- Optional cleanup:
-- DROP USER 'college_user'@'localhost';
```

EXPERIMENT 6: SQL QUERIES AND AGGREGATE FUNCTIONS

MySQL 8.0+ • Beginner-friendly example • Run in MySQL Workbench

```
-- Experiment 6: Basic SELECT and aggregate functions
USE college;

SELECT * FROM student;

SELECT student_name, age
FROM student
WHERE age >= 20;

SELECT DISTINCT dept_id
FROM student;

SELECT *
FROM student
ORDER BY student_name;

SELECT COUNT(*) AS total_students
FROM student;

SELECT AVG(age) AS average_age
FROM student;

SELECT MIN(age) AS youngest
FROM student;

SELECT MAX(age) AS oldest
FROM student;

SELECT dept_id, COUNT(*) AS student_count
FROM student
GROUP BY dept_id;

SELECT dept_id, AVG(age) AS average_age
FROM student
GROUP BY dept_id
HAVING AVG(age) >= 20;
```

EXPERIMENT 7: JOIN OPERATIONS

MySQL 8.0+ • Beginner-friendly example • Run in MySQL Workbench

```
-- Experiment 7: Join operations
USE college;

-- INNER JOIN / Equijoin
SELECT student.student_name, department.dept_name
FROM student
INNER JOIN department
ON student.dept_id = department.dept_id;

-- Non-equijoin
SELECT student.student_name, student.age, department.dept_name
FROM student
JOIN department
ON student.age >= 20;

-- Self-join
SELECT a.student_name AS student1, b.student_name AS student2
FROM student a
JOIN student b
ON a.dept_id = b.dept_id
AND a.roll_no < b.roll_no;

-- LEFT OUTER JOIN
SELECT student.student_name, department.dept_name
FROM student
LEFT JOIN department
ON student.dept_id = department.dept_id;

-- RIGHT OUTER JOIN
SELECT student.student_name, department.dept_name
FROM student
RIGHT JOIN department
ON student.dept_id = department.dept_id;

-- CROSS JOIN
SELECT student.student_name, department.dept_name
FROM student
CROSS JOIN department;
```

EXPERIMENT 8: VIEWS AND TRIGGERS

MySQL 8.0+ • Beginner-friendly example • Run in MySQL Workbench

```
-- Experiment 8: Create and query a view
USE college;

CREATE VIEW student_view AS
SELECT roll_no, student_name, dept_id
FROM student;

SELECT * FROM student_view;

-- Create a log table
CREATE TABLE student_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    action_name VARCHAR(20),
    roll_no INT,
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- AFTER INSERT trigger
DELIMITER //
CREATE TRIGGER student_after_insert
AFTER INSERT ON student
FOR EACH ROW
BEGIN
    INSERT INTO student_log (action_name, roll_no)
    VALUES ('INSERT', NEW.roll_no);
END//

-- AFTER UPDATE trigger
CREATE TRIGGER student_after_update
AFTER UPDATE ON student
FOR EACH ROW
BEGIN
    INSERT INTO student_log (action_name, roll_no)
    VALUES ('UPDATE', NEW.roll_no);
END//

-- AFTER DELETE trigger
CREATE TRIGGER student_after_delete
AFTER DELETE ON student
FOR EACH ROW
BEGIN
    INSERT INTO student_log (action_name, roll_no)
    VALUES ('DELETE', OLD.roll_no);
END//
DELIMITER ;

-- Test and view trigger log
INSERT INTO student (roll_no, student_name, age, dept_id)
VALUES (104, 'Kiran', 20, 1);

UPDATE student SET age = 21 WHERE roll_no = 104;

DELETE FROM student WHERE roll_no = 104;

SELECT * FROM student_log;
SHOW TRIGGERS;
```

EXPERIMENT 9: TRANSACTIONS AND CONCURRENCY CONTROL

MySQL 8.0+ • Beginner-friendly example • Run in MySQL Workbench

```
-- Experiment 9: Transaction and concurrency control
USE college;

START TRANSACTION;

INSERT INTO student (roll_no, student_name, age, dept_id)
VALUES (105, 'Sonal', 20, 1);

UPDATE student
SET age = 21
WHERE roll_no = 105;

SELECT * FROM student WHERE roll_no = 105;

COMMIT;

-- Demonstrate rollback
START TRANSACTION;

UPDATE student
SET age = 25
WHERE roll_no = 105;

SELECT * FROM student WHERE roll_no = 105;

ROLLBACK;

SELECT * FROM student WHERE roll_no = 105;

-- Locking example (run in Session 1)
START TRANSACTION;
SELECT * FROM student
WHERE roll_no = 105
FOR UPDATE;
-- In Session 1, finish with:
-- COMMIT;

-- In Session 2, try updating the same row while Session 1
-- holds the lock. It waits until Session 1 commits.
-- UPDATE student SET age = 22 WHERE roll_no = 105;
```